Experiment No-13 : Minimax Algorithm for Gaming

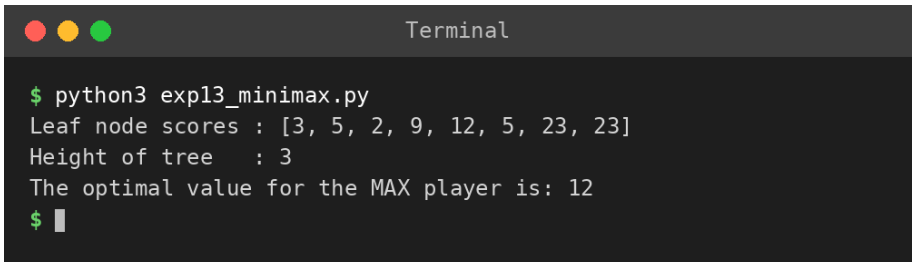
Aim

To write a Python program to implement the Minimax algorithm for gaming in Artificial Intelligence.

Programming Code:

```
import math

def minimax(depth, node_index, is_max, scores, height):
    # Leaf node reached - return its score
    if depth == height:
        return scores[node_index]

    if is_max:
        # MAX player chooses the larger value
        return max(minimax(depth + 1, node_index * 2, False, scores, height),
                   minimax(depth + 1, node_index * 2 + 1, False, scores, height))
    else:
        # MIN player chooses the smaller value
        return min(minimax(depth + 1, node_index * 2, True, scores, height),
                   minimax(depth + 1, node_index * 2 + 1, True, scores, height))


scores = [3, 5, 2, 9, 12, 5, 23, 23]
height = int(math.log2(len(scores)))

print("Leaf node scores :", scores)
print("Height of tree   :", height)
print("The optimal value for the MAX player is:", minimax(0, 0, True, scores, height))
```

OUTPUT:



Expected Output

- The leaf node scores of the game tree are printed: [3, 5, 2, 9, 12, 5, 23, 23].
- The height of the tree is displayed as 3, because 8 leaf nodes give $2^3 = 8$.
- The MAX nodes at depth 2 select 5, 9, 12 and 23, and the MIN nodes at depth 1 then select 5 and 12.
- The MAX player at the root chooses the larger of 5 and 12, so the optimal value is 12.
- The last line of the output is: The optimal value for the MAX player is: 12.

Result

Thus, the Minimax algorithm was implemented successfully in Python. It searches the complete game tree and selects the move that maximizes the player's score while assuming that the opponent always plays to minimize it.